Builder

Building the intuition

Before any code, let's feel the pain that Builder was invented to solve. The pattern exists because object construction — when done naively — becomes one of the most error-prone parts of any codebase.

THE PROBLEM — TELESCOPING CONSTRUCTORS

You're building a backend that sends emails. An email has many possible fields. Your first instinct is a constructor:

```
// Version 1 — simple, only required fields
public Email(String to, String subject, String body) { }

// Version 2 — add CC
public Email(String to, String subject, String body, String cc) { }

// Version 3 — add BCC
public Email(String to, String subject, String body, String cc, String bcc) { }

// Version 4 — add reply-to, priority, template, attachments...
public Email(String to, String subject, String body, String cc,
             String bcc, String replyTo, int priority,
             String template, List<File> attachments) { }
```

This is called the **telescoping constructor anti-pattern**. The constructor grows with every new optional field. Callers must pass `null` for fields they don't need. And this call site — `new Email("a@b.com", "Hi", "Body", null, null, null, 1, null, null)` — is completely unreadable. What does that 7th argument mean? You have to count parameters every time.

THE ANALOGY — A CUSTOM BURGER ORDER

> Think of ordering a custom burger. You don't hand the chef a 15-item list of constructor arguments in a fixed order. You tell them step by step: *I want a sesame bun. Add the beef patty. Yes to cheese. No lettuce. Extra pickles. No sauce.* When you say "done", the chef hands you the finished burger — complete, valid, exactly as specified.
>
> The Builder pattern is that order process. Each step is a named setter on a builder object. At the end, you call `build()` and get a fully validated, immutable product — assembled in any order you like, with only the fields you care about.

BEFORE VS. AFTER — THE SAME EMAIL, TWO WAYS

Without Builder

```
new Email(to, subj, body, null, null,
null, 1, null, null)
```

9 args — which is which? nulls everywhere

Unreadable. Error-prone. No validation gate.
Mutable state if setters used instead.

With Builder

```
Email.builder()
.to("user@example.com")
.subject("Order confirmed")
.body("Your order shipped.")
.priority(HIGH)
.build()
```

Self-documenting. Validates on build().
Result is immutable. Only needed fields set.

> The key insight: Builder *separates construction from representation*. The construction process lives in a dedicated object (the Builder). The result (the Product) is handed back fully formed and validated. Your business code is readable, your Product is immutable, and validation happens exactly once — at `build()`.

The four components

The classic Builder pattern has four roles. In real backend Java, you'll use the simplified inner-class version (which merges two of them). Know both — the classic teaches you the theory; the inner-class version is what you actually write.

class

Product

The complex object being built. It has many fields — some required, many optional. Crucially, it has *no public setters* after construction. All fields are set via the builder and the Product becomes immutable once `build()` is called.

Example: `Email` , `HttpRequest` , `QueryCriteria` .

interface

Builder (interface)

Declares all the step methods that set fields on the eventual product, plus the final `build()` method that returns the Product. Having an interface here is optional in the simple case — it becomes valuable when you need multiple builder implementations for the same product.

Example: `EmailBuilder` interface.

class

Concrete Builder

Implements the Builder interface. Holds the intermediate state during construction. Each step method sets a field and returns `this` — enabling the fluent method-chaining API. The `build()` method validates and returns the finished Product.

Example: `HtmlEmailBuilder` , `PlainEmailBuilder` .

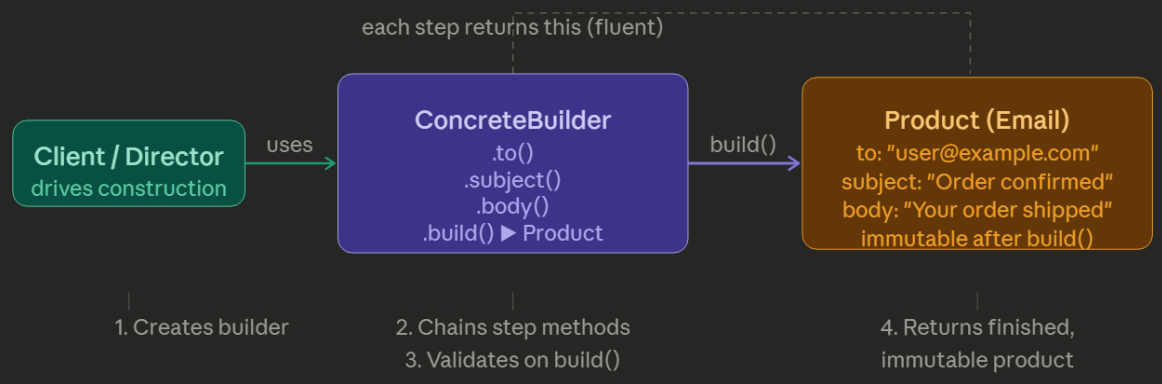
class

Director (optional)

Orchestrates the builder steps in a specific sequence to produce a standard, pre-defined product variant. The Director knows *what* to build; the Builder knows *how*. In practice (and in most Java code), the Director is often omitted — the caller drives the builder directly.

Example: `WelcomeEmailDirector` that always sets the same template fields.

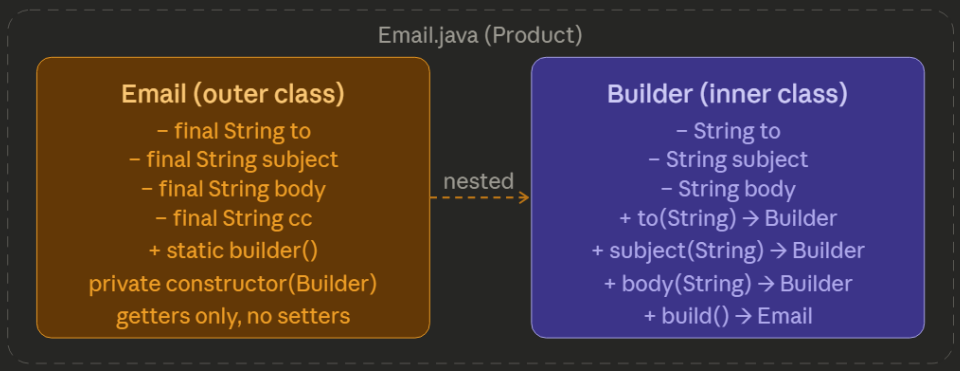
HOW THE ROLES CONNECT AT RUNTIME



> The fluent API works because every step method returns `this` — the same builder instance. So calls chain naturally: `builder.to(...).subject(...).body(...).build()`. Nothing is copied or cloned between steps — the builder accumulates state, then stamps it into the final Product on `build()`.

THE SIMPLIFIED INNER-CLASS VERSION (MOST COMMON IN JAVA)

In practice, the Builder interface and Concrete Builder are collapsed into a single *static inner class* directly inside the Product. This is the version Lombok generates with `@Builder`, and it's the version you'll write by hand most often. The Director is dropped — the caller chains steps directly. This is simpler and perfectly adequate for 95% of backend use cases.



Use cases in backend development

Builder is not just for emails. It appears constantly across backend code — anywhere a complex object needs to be assembled with clarity. These are the real places you'll encounter or write it.

- **HTTP client requests**
  Java's `HttpRequest.newBuilder()` , OkHttp's `Request.Builder` , Spring's `WebClient.RequestBodySpec` — all use Builder. An HTTP request has URL, method, headers, body, timeout, retry policy, auth — many optional, many interdependent. Builder keeps this readable and validated.

- **Database query / filter criteria objects**
  Instead of a service method like `findProducts(String kw, String cat, BigDecimal min, BigDecimal max, Boolean stock, String sort, int page)` — which is unreadable and forces nulls for unused params — build a `ProductQueryCriteria` object. The repository receives one clean, self-describing object. This is the dominant pattern in well-structured Spring Data repositories.

- **Email and notification message construction**
  Composing transactional emails, Slack messages, push notifications — all have a core of required fields surrounded by many optional ones (CC, BCC, reply-to, template variables, attachments, priority). Builder ensures you never send a malformed message; validation gates at `build()` catch missing required fields before anything is dispatched.

- **API response / DTO assembly**
  When constructing response DTOs from multiple domain objects — user data + account data + permission data + preference data — a Builder lets you set fields in any order as you gather them across service calls, then stamp out the final immutable response object. Avoids ugly intermediate mutable maps or half-populated objects.

- **gRPC / Protobuf message construction**
  Protobuf generates Builder classes automatically for every message type. Every gRPC call in Java uses the Builder API: `OrderResponse.newBuilder().setOrderId(...).setStatus(...).addAllItems(...).build()` . You're already using Builder every time you work with gRPC — now you know its name.

- **Test data setup (Test Builder pattern)**
  In unit and integration tests, you need objects in specific states. Builder shines here: create a base valid object with defaults, then use `toBuilder()` to override only the field relevant to each test case. This gives you clean, self-documenting test setup without sprawling fixtures or fragile helper methods.

- **Configuration objects (app startup)**
  When building infrastructure configuration objects — database pool config, Redis client config, Kafka producer config — the configuration space is huge and most fields have sensible defaults. Builder lets you specify only what differs from defaults. Spring Boot's auto-configuration beans are assembled this way under the hood.

Pros & cons

Builder is one of the most universally loved patterns because its benefits are felt immediately in code readability. But it does have genuine costs — here's how a senior engineer weighs them.

Pros

- Eliminates telescoping constructors — no more counting positional arguments
- Self-documenting call sites — `.priority(1)` is clear; the 7th positional argument is not
- Enables immutability — Product fields are `final`, object is safe across threads
- Single validation gate — required fields checked once, at `build()`
- Order-independent construction — set fields in any sequence that suits the caller
- Sensible defaults — optional fields get defaults without overloaded constructors
- Lombok kills the boilerplate — `@Builder` makes the cost of adoption near zero

Cons

- More code (without Lombok) — doubles the field declarations across Product and Builder
- Mutable builder state — the Builder itself is mutable during construction; don't share across threads
- Overkill for simple objects — a class with 2-3 fields needs a plain constructor, not a Builder
- Partial builds are silent traps — if you forget `.build()`, you hold a Builder, not a Product; the compiler won't tell you
- Validation in `build()` is not compile-time — missing a required field is a runtime error, not a compile error

DON'T USE BUILDER WHEN

The object has 3 or fewer fields, all required. A plain constructor is cleaner: `new Money(100, "USD")` is more readable than a three-step builder chain. Builder adds ceremony where none is needed.

USE BUILDER WHEN

The object has 4+ fields, especially with optional ones. The field distinction between required and optional matters. You want immutability. You need a validation gate. Lombok is available (making the cost essentially zero).

BUILDER VS. RELATED PATTERNS — THE DISTINCTIONS THAT TRIP PEOPLE UP

Builder vs. Factory Method

Factory Method answers *"which type?"* — it selects among implementations of an interface at runtime. Builder answers *"how is this one type configured?"* — it assembles a single complex object step by step. They often work together: a Factory can use a Builder internally to assemble each product variant.

Builder vs. Prototype (clone)

Prototype creates a new object by cloning an existing one. Builder creates from scratch. Use Prototype when you have a good base instance and want slight variations. Use Builder when you're constructing from nothing with arbitrary configuration. Lombok's `toBuilder()` is a hybrid — it seeds a Builder from an existing object.

Builder vs. plain setters

Setters allow modification after construction — the object is forever mutable and can be in an invalid state at any point. Builder's whole value proposition is that the object only ever exists in a valid, immutable state. If you use setters, you give up validation guarantees and thread safety.

THE SENIOR ENGINEER'S RULE OF THUMB

> **If your constructor has more than 3 parameters, or any of them are nullable/optional — reach for Builder.** In Spring Boot projects with Lombok, the cost of adding `@Builder` is a single annotation. The benefit is permanent readability and immutability. The calculus almost always favors Builder for any non-trivial domain or DTO object. When you see a method call with 5+ arguments and half of them are `null`, that's your signal that someone should have used a Builder.